COS 433: Cryptography

Instructor: Alex Lombardi, Office Hours Monday 4:15pm

TAs: Lacey Delucia, Fangqi Dong, Frederick Qiu

(Office Hours Tuesday 4:30pm, Thursday 3pm)

Email (use this to reach us): cos433staff@princeton.edu

Course Website:

princeton-cos-433.github.io

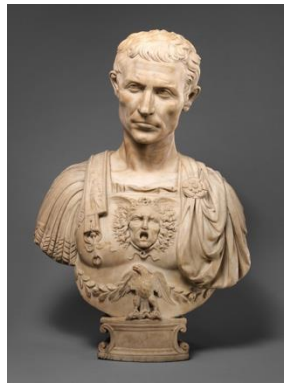
What is cryptography?

What is cryptography?

Pre-1950: the art of secret writing

“Yhql ylgl ylfl”

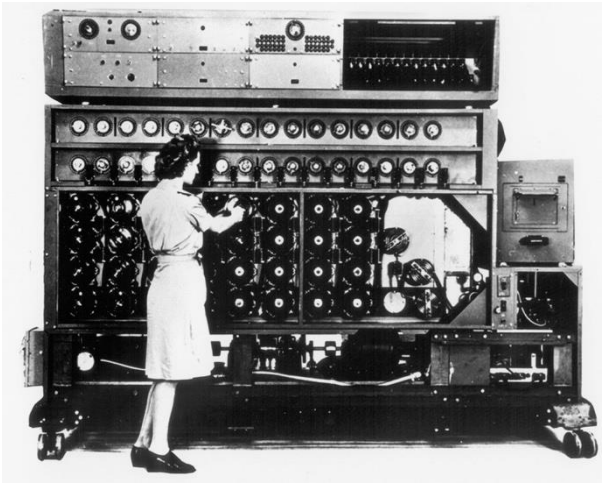
“Veni vidi vici”



"If he had anything confidential to say, he wrote it in cipher, that is, by so changing the order of the letters of the alphabet, that not a word could be made out. If anyone wishes to decipher these, and get at their meaning, he must substitute the fourth letter of the alphabet, namely D, for A, and so with the others."

—*Suetonius*, *Life of Julius Caesar* *56*

1900-1950: one of the first applications of (primitive) computers



What is cryptography?

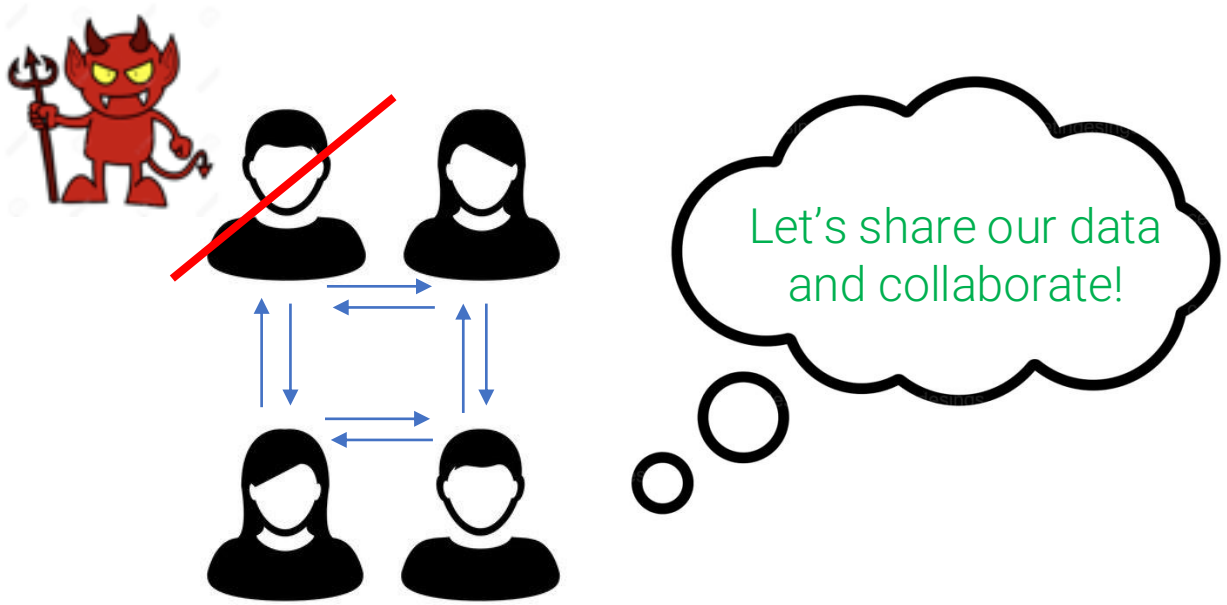
Modern definition:

> The design and analysis of protocols that resist adversarial behavior.

What is cryptography?

Modern definition:

The design and analysis of protocols that resist adversarial behavior.



Theorem:  are protected from 

What is cryptography?

Modern definition:

The design and analysis of protocols that resist adversarial behavior.

Another definition:

The algorithmic and mathematical foundations of secure communication and computation.

What is cryptography any good for?

Authorize an online transaction…

without identity theft

Perform a massive, private computation…

without running it locally

Course Information

Course Information

What you need in order to succeed:

- Comfort writing and understanding mathematical proofs at the level of COS 330
- Basic probability (expected value, variance)
- Asymptotic running time of an algorithm (like $O(n^2)$ vs. $O(2^n)$)

Helpful background: number theory (modular arithmetic), algorithms, NP hardness.

This course may be challenging! Come to office hours, ask questions (about the problem sets, lectures, etc.) on Ed, collaborate with your classmates.

Assignments

This course will have the following graded assignments:

- **Problem Set 0:** A short, one-week problem set at the beginning of the semester to help you calibrate your background.
- **Problem Sets 1–5:** Regular-length, biweekly problem sets.
- **Problem Set 6:** A cumulative problem set that will serve as your final.

Grading: Your assignments will be graded in two parts:

1. Your written work.
2. Check-ins with the course staff. After Problem Sets 0, 3, and 6, you'll have a short individual meeting about your recent solutions. In the meeting, we will ask you to present some of your solutions and explain your thought process.

Submit solutions on Gradescope. LaTeX typesetting strongly encouraged.

Check-in 1: Monday 9/14 and Tuesday 9/15 (sign-up link on problem set 0)

Late Submission and Homework Grading Policy

You may take up to two late days for any problem set. These extensions are granted automatically (no need to ask permission!). We cannot accept any submissions that are more than two days late. If you have an extenuating circumstance such as a medical emergency, please reach out to the course staff.

Problem Set 0 is an exception: we cannot accept late submissions outside of extenuating circumstances.

To encourage timely submission, two bonus points (out of 100) will be granted to submissions that are on time.

Collaboration Policy

You are free (and strongly encouraged!) to collaborate with each other on Problem Sets 0–5. Please adhere to the following policies regarding collaboration and attribution.

- Individually write up problem set solutions in your own words. You may not share write-ups with your collaborators.
- List all student collaborators and consulted outside sources (e.g. textbooks, course notes) in your problem set solutions.
- You may not collaborate with other students on Problem Set 6.

AI Use Policy

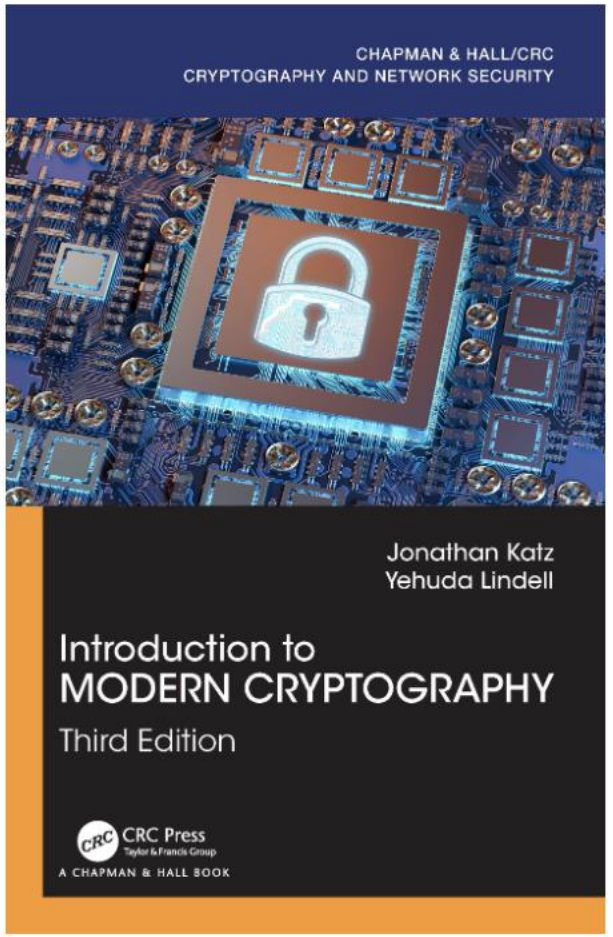
AI tools can be useful learning aids, and you are welcome to use them to explore and review general course material. You may also use AI to generate visual aids, such as diagrams, in your assignments. However, please follow two rules:

- Do not ask AI tools for solutions to assigned problems.
- Submit only writing that is entirely your own; do not submit any AI-generated text.

Feel free to reach out with any questions!

Optional Textbook

"Katz-Lindell"



Please interrupt me if you are confused!

Part I: Secret-Key Encryption and Authentication

Secret-Key Encryption

Eve

Alice

"ciphertext" $\mathbf{ct}$

Bob

"secret key" k

"plaintext" message m

"secret key" k

receives m

Key point: must leverage some sort of information asymmetry between Bob and Eve

Secret-Key Encryption: Syntax

k, m

$$\mathrm{ct} = \mathrm{Enc}(k, m)$$

k

$$m' = \mathrm{Dec}(k, \mathrm{ct})$$

Definition: A secret-key encryption scheme $(\mathbf{Enc}, \mathbf{Dec})$ consists of two algorithms:

- $\mathrm{Enc}(k, m)$ takes a key $k \in \mathcal{K}$ and message $m \in \mathcal{M}$ and outputs a ciphertext $\mathrm{ct} \in \mathcal{C}$
- $\mathrm{Dec}(k, \mathrm{ct})$ takes as input a key and ciphertext and outputs a message m'

Secret-Key Encryption: Syntax

k, m

$$\mathrm{ct} = \mathrm{Enc}(k, m)$$

k

$$m' = \mathrm{Dec}(k, \mathrm{ct})$$

Definition: A secret-key encryption scheme $(\mathbf{Enc}, \mathbf{Dec})$ consists of two algorithms:

- $\mathrm{Enc}(k, m; r)$ takes a key $k \in \mathcal{K}$ and message $m \in \mathcal{M}$ and outputs a ciphertext $\mathbf{ct} \in \mathcal{C}$
- $\mathrm{Dec}(k, \mathrm{ct})$ takes as input a key and ciphertext and outputs a message m'

Disclaimer: **encryption** can either be **deterministic** or randomized

Secret-Key Encryption: Syntax

k, m

$\mathrm{ct} = \mathrm{Enc}(k, m)$

k

$m' = \mathrm{Dec}(k, \mathrm{ct})$

Definition: A secret-key encryption scheme $(\mathbf{Enc}, \mathbf{Dec})$ consists of two algorithms:

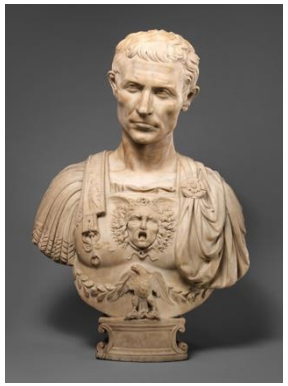
- $\mathrm{Enc}(k, m)$ takes a key $k \in \mathcal{K}$ and message $m \in \mathcal{M}$ and outputs a ciphertext $\mathrm{ct} \in \mathcal{C}$
- $\mathrm{Dec}(k, \mathrm{ct})$ takes as input a key and ciphertext and outputs a message m'

A secret-key encryption scheme $(\mathbf{Enc}, \mathbf{Dec})$ is correct if for all $k \in \mathcal{K}, m \in \mathcal{M}$

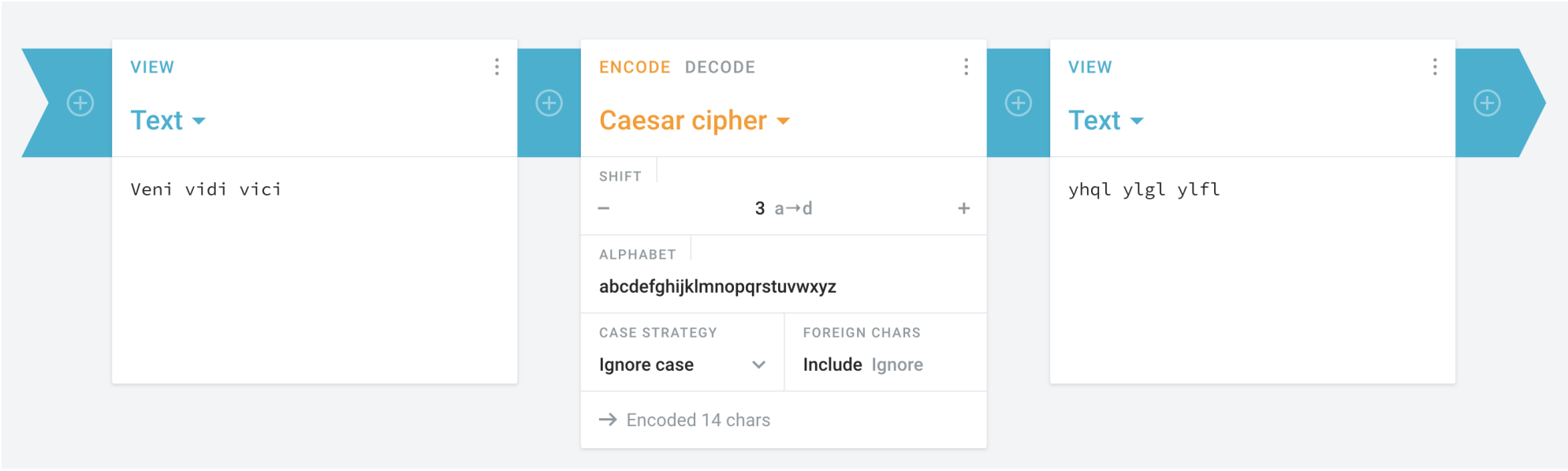
$$\mathrm{Dec}\big(k, \mathrm{Enc}(k, m)\big) = m$$

Secret-Key Encryption

Historical examples: https://cryptii.com/



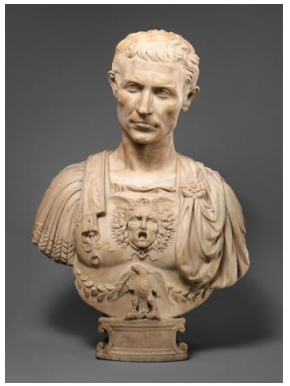
- Caesar cipher: map every character $\mathbf{ch}$ of the English (or Latin) alphabet to $\mathbf{ch} + k$ (with wrap-around). $k \in \mathbb{Z}, 0 \leq k \leq 25$



Secret-Key Encryption

Historical examples: https://cryptii.com/

- Caesar cipher: map every character $\mathbf{ch}$ of the English (or Latin) alphabet to $\mathbf{ch} + k$ (with wrap-around). $k \in \mathbb{Z}, 0 \leq k \leq 25$
  - Suppose you know that a ciphertext $\mathbf{ct}$ is encrypted via Caesar. Can you, Eve, break the scheme?

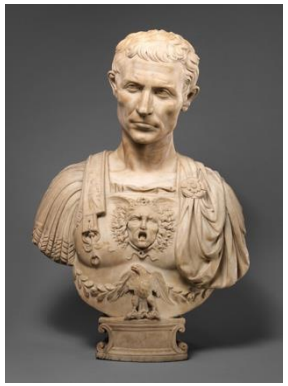


"Yhql ylgl ylfl"

Brute force attack: compute $\mathbf{Dec}(k', \mathbf{ct})$ for all 26 possible k'.

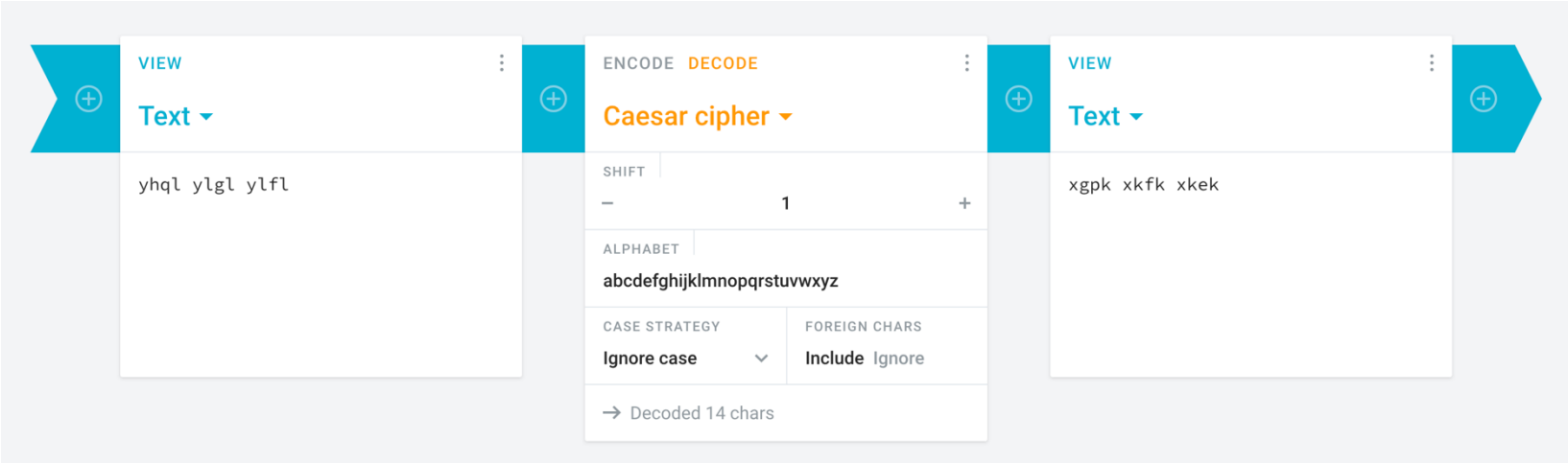
Secret-Key Encryption

Historical examples: https://cryptii.com/



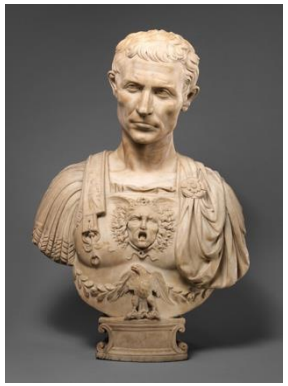
"Yhql ylgl ylfl"

- Caesar cipher: map every character $\mathbf{ch}$ of the English (or Latin) alphabet to $\mathbf{ch} + k$ (with wrap-around). $k \in \mathbb{Z}, 0 \leq k \leq 25$
  - Suppose you know that a ciphertext $\mathbf{ct}$ is encrypted via Caesar. Can you, Eve, break the scheme?



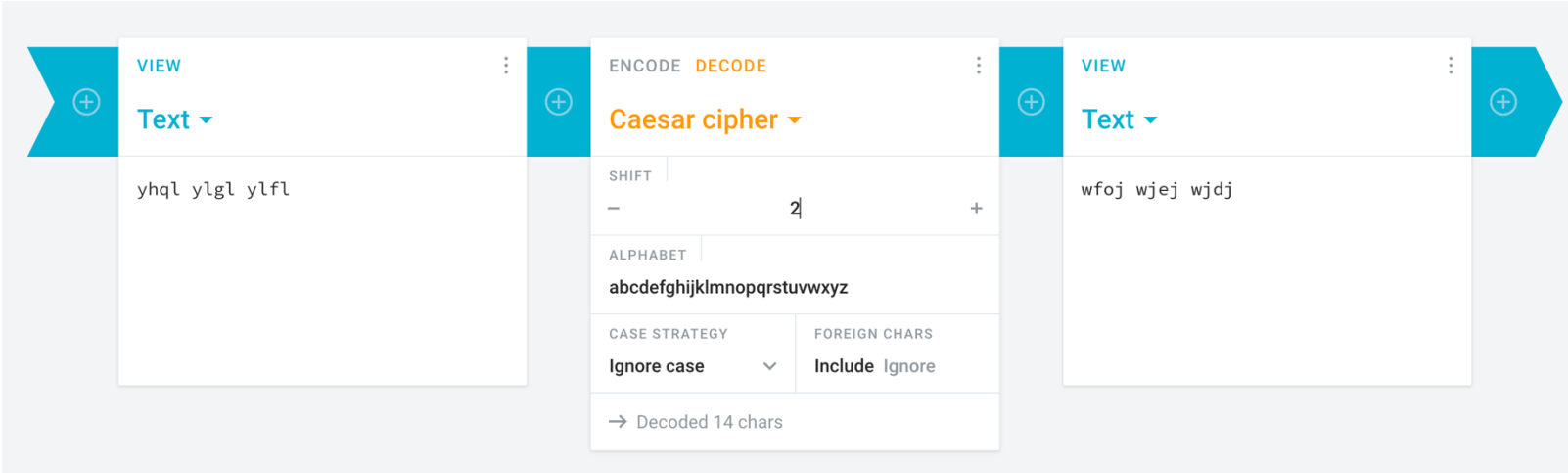
Secret-Key Encryption

Historical examples: https://cryptii.com/



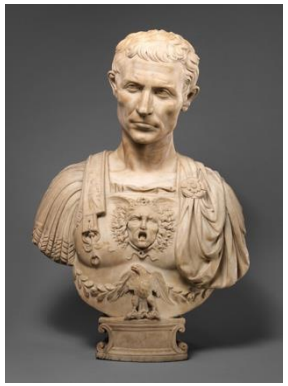
"Yhql ylgl ylfl"

- Caesar cipher: map every character $\mathbf{ch}$ of the English (or Latin) alphabet to $\mathbf{ch} + k$ (with wrap-around). $k \in \mathbb{Z}, 0 \leq k \leq 25$
  - Suppose you know that a ciphertext $\mathbf{ct}$ is encrypted via Caesar. Can you, Eve, break the scheme?



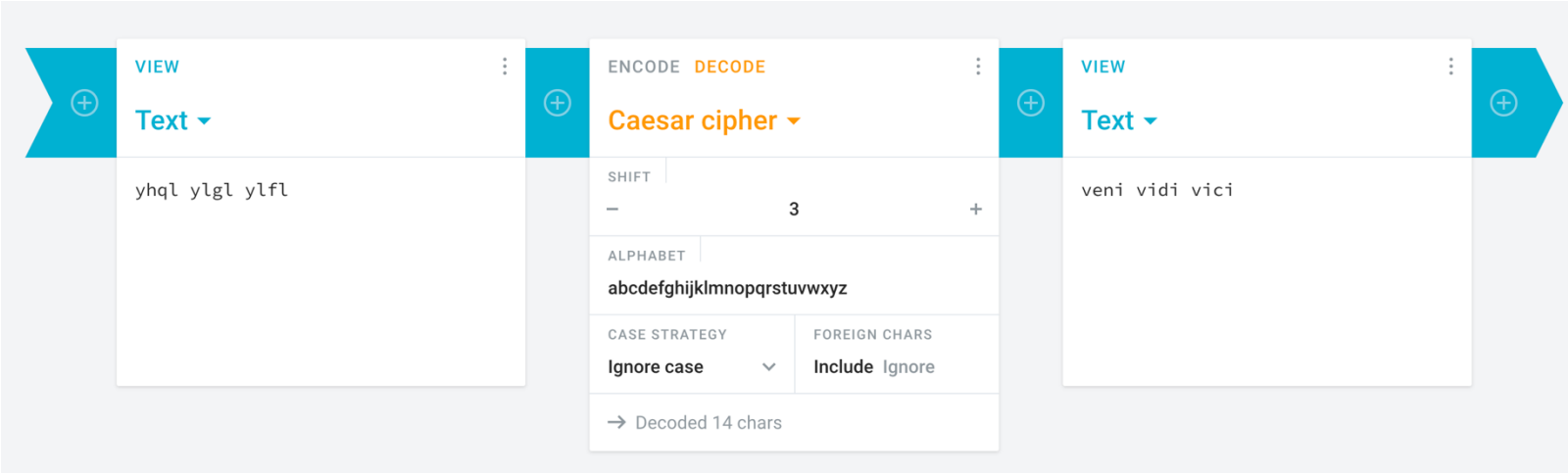
Secret-Key Encryption

Historical examples: https://cryptii.com/



"Yhql ylgl ylfl"

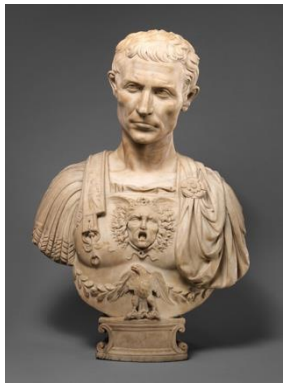
- Caesar cipher: map every character $\mathbf{ch}$ of the English (or Latin) alphabet to $\mathbf{ch} + k$ (with wrap-around). $k \in \mathbb{Z}, 0 \leq k \leq 25$
  - Suppose you know that a ciphertext $\mathbf{ct}$ is encrypted via Caesar. Can you, Eve, break the scheme?



Secret-Key Encryption

Historical examples: https://cryptii.com/

- Caesar cipher: map every character $\mathbf{ch}$ of the English (or Latin) alphabet to $\mathbf{ch} + k$ (with wrap-around). $k \in \mathbb{Z}, 0 \leq k \leq 25$
  - Suppose you know that a ciphertext $\mathbf{ct}$ is encrypted via Caesar. Can you, Eve, break the scheme?



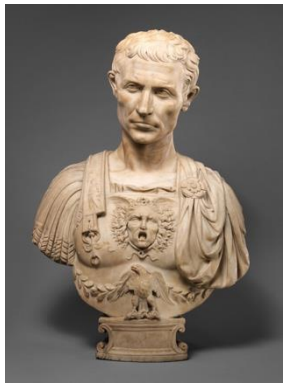
"Yhql ylgl ylfl"

Brute force attack: compute $\mathbf{Dec}(k', \mathbf{ct})$ for all 26 possible k'.

If m is an English language phrase or sentence, most likely only one makes any sense.

Secret-Key Encryption

Historical examples: https://cryptii.com/



"Yhql ylgl ylfl"

- Caesar cipher: map every character $\mathbf{ch}$ of the English (or Latin) alphabet to $\mathbf{ch} + k$ (with wrap-around). $k \in \mathbb{Z}, 0 \leq k \leq 25$
  - Suppose you know that a ciphertext $\mathbf{ct}$ is encrypted via Caesar. Can you, Eve, break the scheme?

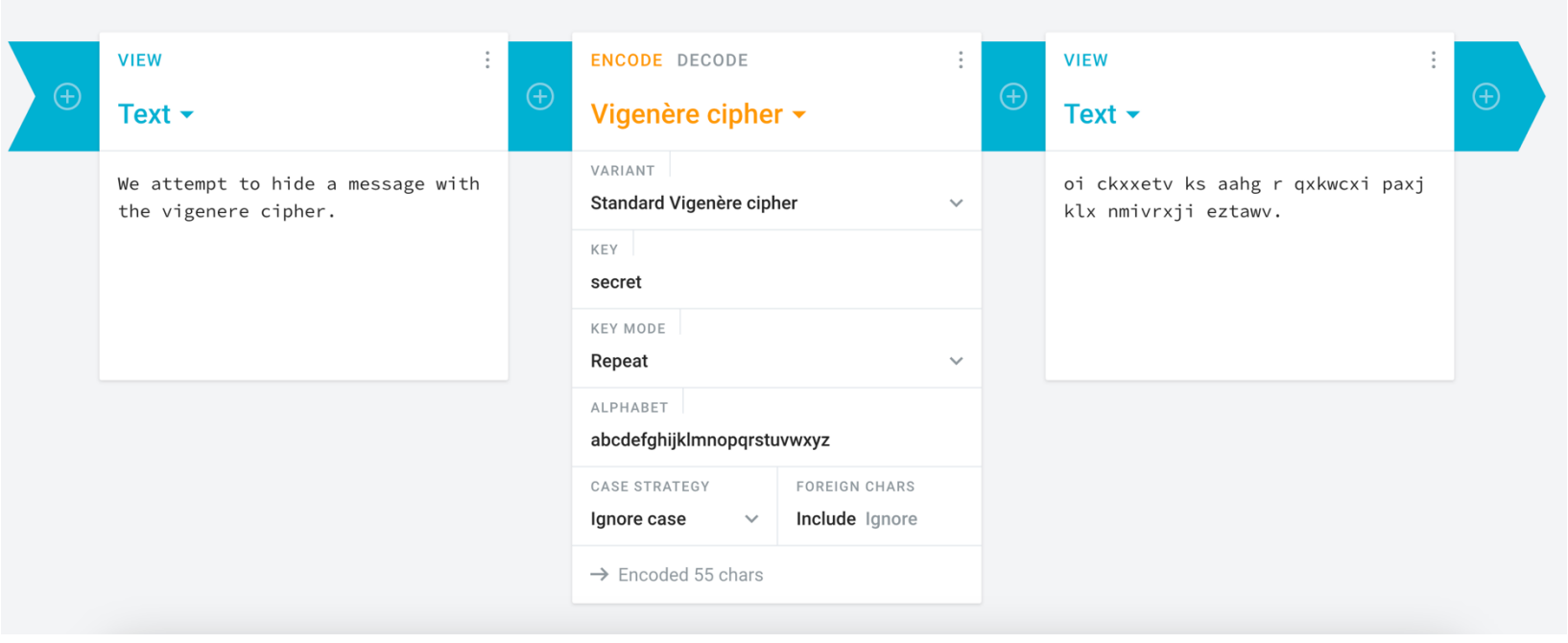
The only thing protecting you when you use a Caesar cipher is obscurity (Eve might not know you are using a Caesar cipher).

"**Security through obscurity**" -- unreliable

Secret-Key Encryption

Historical examples: https://cryptii.com/

- Vigenère cipher: key is a character string $k_1 \ldots k_n$ (often a word)
  - Combines n Caesar ciphers by alternating between the k_i, letter by letter



Secret-Key Encryption

Historical examples: https://cryptii.com/

- Vigenère cipher: key is a character string $k_1 \ldots k_n$ (often a word)
  - Combines n Caesar ciphers by alternating between the k_i, letter by letter

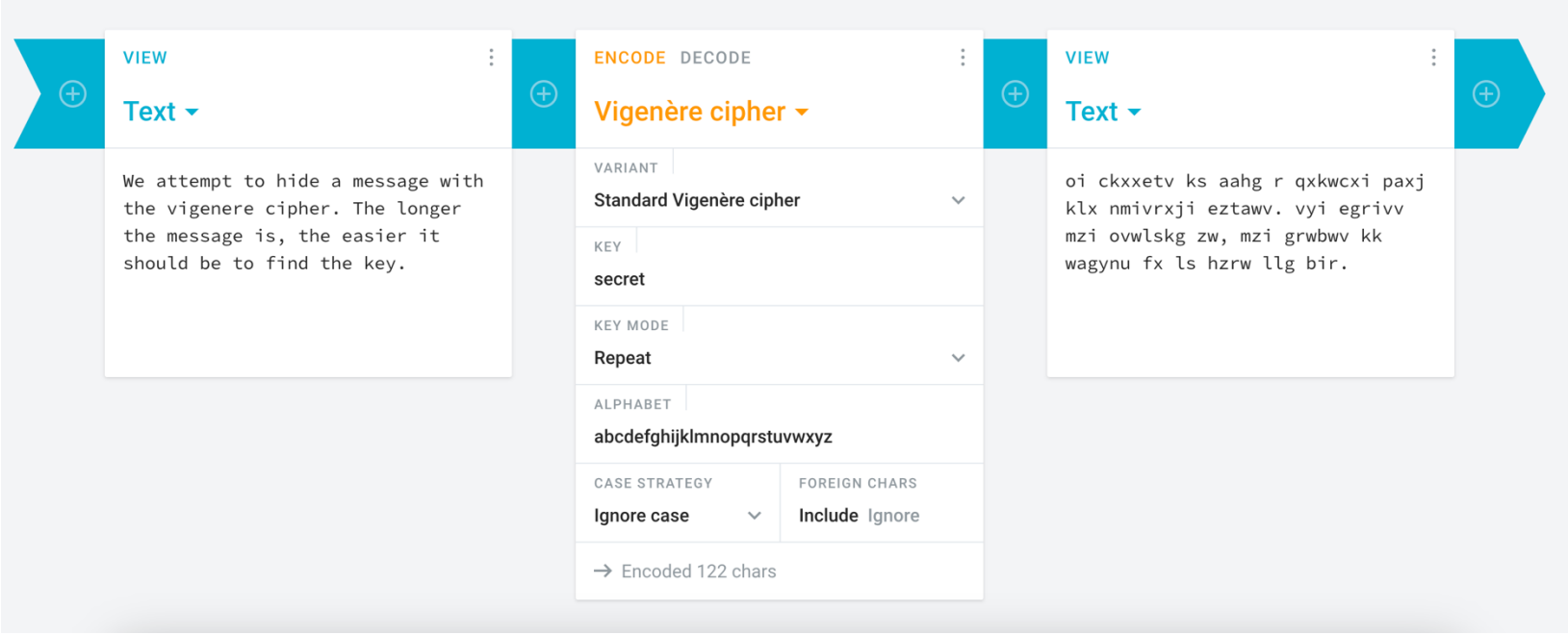
Suppose you know that a ciphertext $\mathbf{ct}$ is encrypted via Vigenère. Can you, Eve, break the scheme?

Much more challenging! Brute force no longer works if n is large (26^n keys!) .

Secret-Key Encryption

Historical examples: https://cryptii.com/

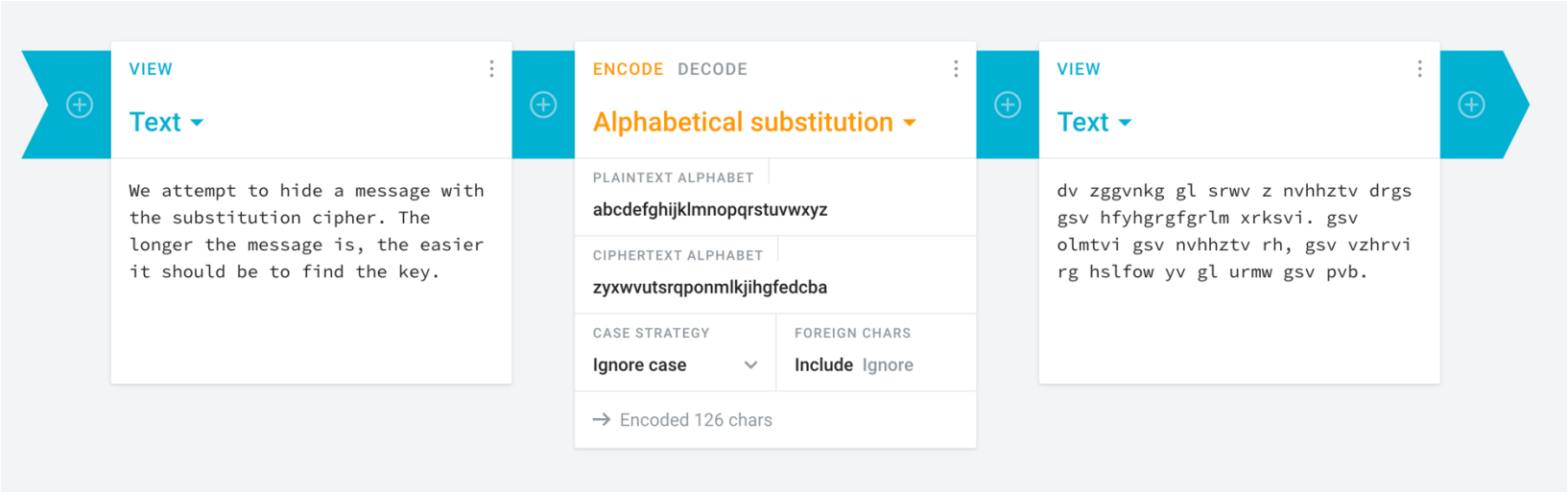
- Vigenère cipher: key is a character string $k_1 \ldots k_n$ (often a word)
  - Combines n Caesar ciphers by alternating between the k_i, letter by letter



Secret-Key Encryption

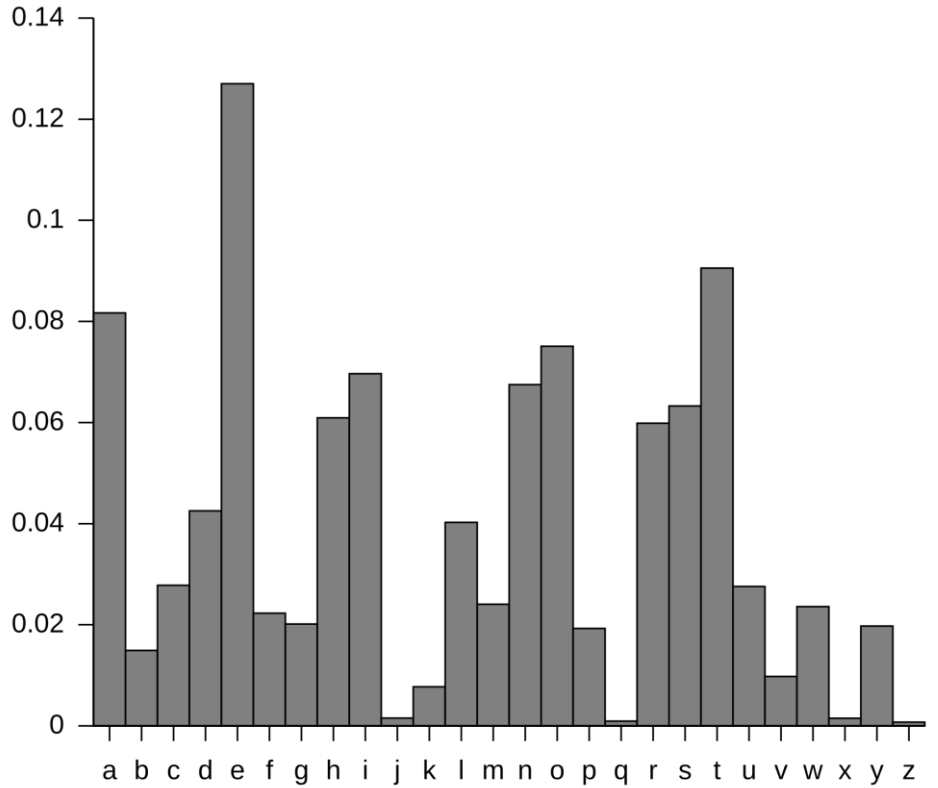
Historical examples: https://cryptii.com/

- Substitution cipher: key is a permutation π of the alphabet. Map every character $\mathbf{ch}$ to $\pi(\mathbf{ch})$. $26! \approx 4 \cdot 10^{26}$ keys, cannot brute force by hand.



Breaking Substitution Ciphers

This is a graph of the frequency of each letter in written English.



Given a sufficiently long ciphertext, one can generate this graph ("frequency analysis") and greatly narrow down what π is.

Breaking the Vigenère Cipher

Vigenère cipher: key is a character string $k_1 \ldots k_n$ (often a word)

- Combines n Caesar ciphers by alternating between the k_i, letter by letter

Attack:

1. Brute force over the possible key lengths (e.g. start from $n = 1$ and count up)
2. Divide the ciphertext characters into n groups (following the Vigenère pattern)
3. Perform frequency analysis on the n groups separately!

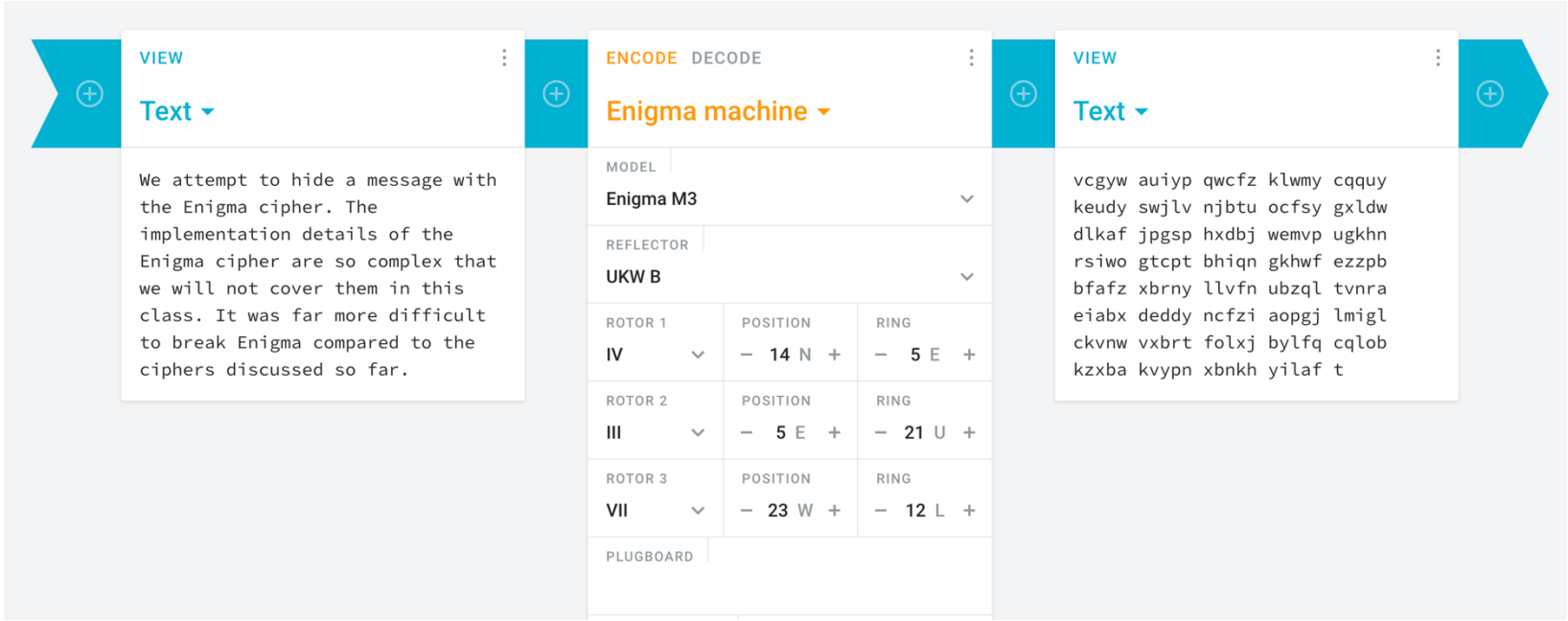
Breaking the Vigenère Cipher

- Our attack did not make use of the fact that each of the n parts are encrypted with a Caesar cipher; would have worked even if they were substitution ciphers.
- This breaks the more general class of "poly-alphabetic ciphers"
- However, this does require that Eve can obtain a sufficiently long ciphertext encrypting "natural language."

Secret-Key Encryption

Enigma cipher: extremely complex variant of poly-alphabetic cipher.

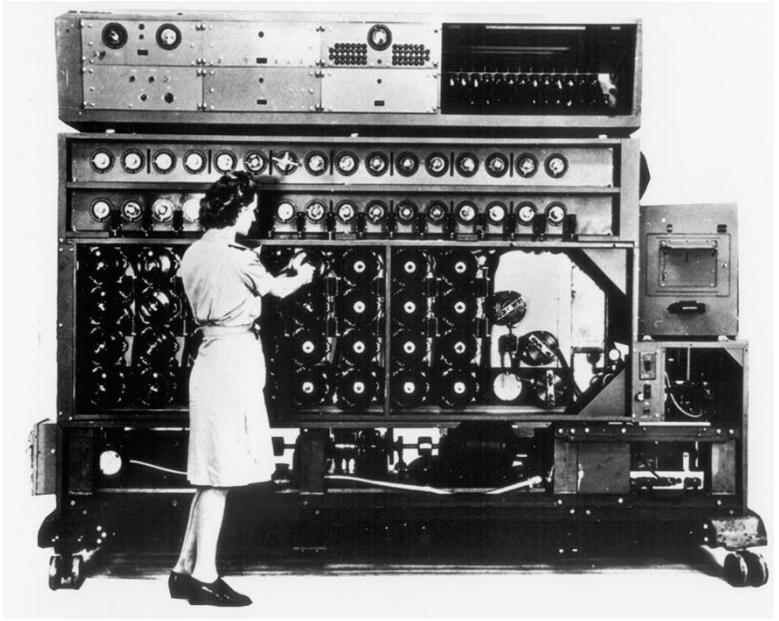
- Utilized by the Axis powers in WWII
- Theoretically broken given enormously long messages, but their protocols called for changing keys too frequently for this to happen.



Secret-Key Encryption

Enigma cipher: extremely complex variant of poly-alphabetic cipher.

- Broken in practice in several cycles (the Germans employed countermeasures)
- Contributing factors: obtaining copies of the machine, exploiting small flaws to narrow key search space, user error (bad keys), 2^{20} time computer search.

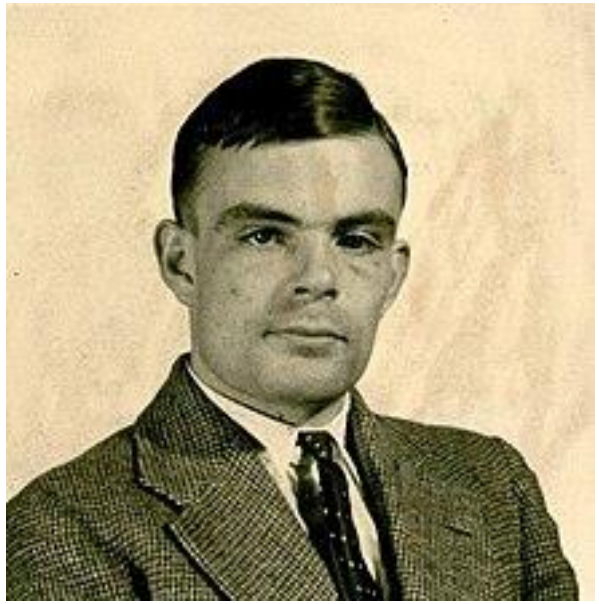


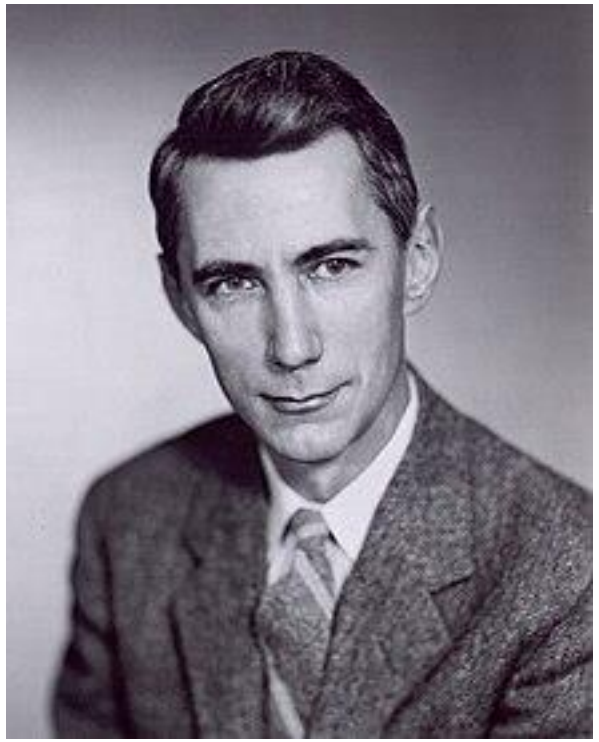
Secret-Key Encryption

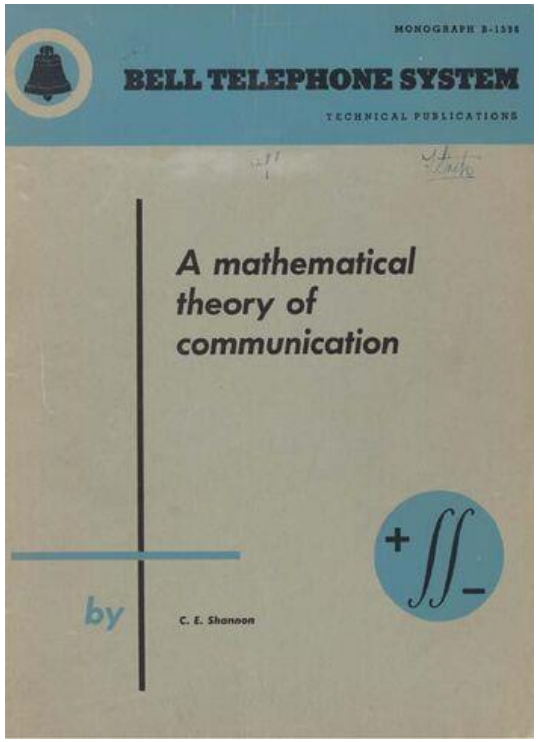
Takeaways:

- It can be really hard to intuit if an encryption scheme is truly secure!
- Security through obscurity is a bad plan in the long run.
  - **Kerckhoffs' principle** (1883): the security of a cryptosystem shouldn't rely on the secrecy of the algorithm (only the key)
  - NSA: "The standard assumption there was that serial number one of any new device was delivered to the Kremlin."
- What we need is a formal framework for defining and proving security

Secret-Key Encryption







- What we need is a formal framework for defining and proving security

Information/probability theory

Secret-Key Encryption: Security

k, m

$\mathrm{ct} = \mathrm{Enc}(k, m)$

k

Adversary Model: Eve is "passive," reads communication but does not interfere.

Insight: Alice and Bob should sample k uniformly at random from $\mathcal{K}$.

Major simplification: focus on the case of encrypting a *single message*.

How should we define security?

Secret-Key Encryption: Security

k, m

$\mathrm{ct} = \mathrm{Enc}(k, m)$

k

Proposal 1: Scheme is secure if the key is completely unguessable.

Formally, for all $m \in \mathcal{M}$ and all functions $\mathbf{Eve}: \mathcal{C} \to \mathcal{K}$

$$\Pr_{k \leftarrow \mathcal{K}}\left[\mathrm{Eve}\left(\mathrm{Enc}(k, m)\right) = k\right] = \frac{1}{|\mathcal{K}|}$$

Counterexample: $\mathrm{Enc}(k, m) = m$

Secret-Key Encryption: Security

k, m

$$\mathrm{ct} = \mathrm{Enc}(k, m)$$

k

Proposal 2: Scheme is secure if the message is completely unguessable.

What does that mean?

Let's handle the case of a uniformly random message.

Secret-Key Encryption: Security

k, m

$\mathrm{ct} = \mathrm{Enc}(k, m)$

k

Definition 1: a secret-key encryption scheme $(\mathbf{Enc}, \mathbf{Dec})$ is perfectly secret for uniform messages if for all functions $\mathbf{Eve}: \mathcal{C} \to \mathcal{M}$

$$\Pr_{\substack{k \leftarrow \mathcal{K} \\ m \leftarrow \mathcal{M}}} \left[\mathrm{Eve}\left(\mathrm{Enc}(k, m)\right) = m\right] = \frac{1}{|\mathcal{M}|}$$

Perfect Secrecy

Definition 1: a secret-key encryption scheme $(\mathbf{Enc}, \mathbf{Dec})$ is perfectly secret for uniform messages if for all functions $\mathbf{Eve}: \mathcal{C} \to \mathcal{M}$

$$\Pr_{\substack{k \leftarrow \mathcal{K} \\ m \leftarrow \mathcal{M}}} \left[\mathbf{Eve}\left(\mathbf{Enc}(k, m) \right) = m \right] = \frac{1}{|\mathcal{M}|}$$

Perfect Secrecy

Definition 1': a secret-key encryption scheme $(\mathbf{Enc}, \mathbf{Dec})$ is perfectly secret for uniform messages if for all messages $m^* \in \mathcal{M}$ and all ciphertexts $\mathbf{ct} \in \mathcal{C}$

$$\Pr_{\substack{k \leftarrow \mathcal{K} \\ m \leftarrow \mathcal{M}}} [m = m^* | \, \mathrm{Enc}(k, m) = \mathbf{ct}] = \frac{1}{|\mathcal{M}|}$$

This is equivalent to Definition 1. Why?

The best strategy for $\mathbf{Eve}$ given $\mathbf{ct}$ is to guess the message with the highest conditional probability of occurring.

Perfect Secrecy

Definition 1': a secret-key encryption scheme $(\mathbf{Enc}, \mathbf{Dec})$ is perfectly secret for uniform messages if for all messages $m^* \in \mathcal{M}$ and all ciphertexts $\mathbf{ct} \in \mathcal{C}$

$$\Pr_{\substack{k \leftarrow \mathcal{K} \\ m \leftarrow \mathcal{M}}} [m = m^* | \mathbf{Enc}(k, m) = \mathbf{ct}] = \frac{1}{|\mathcal{M}|}$$

But my messages aren't (uniformly) random!

COS 433: Cryptography

Instructor: Alex Lombardi, Office Hours Monday 4:15pm

TAs: Lacey Delucia, Fangqi Dong, Frederick Qiu (OH TBD)

Email (use this to reach us): cos433staff@princeton.edu

Course Website:

princeton-cos-433.github.io